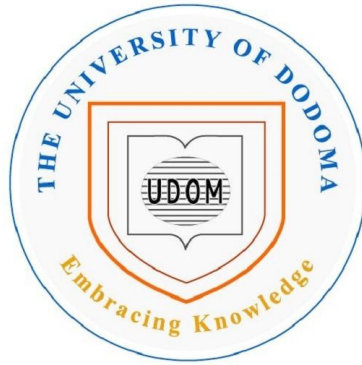


THE UNIVERSITY OF DODOMA.



COLLEGE OF INFORMATICS AND VIRTUAL EDUCATION.

GROUP 5 PROJECT REPORT.

SPI-Based Digital Output Expansion System

MEMBERS.

NAME	REGISTRATION	COURSE
SAID BARAZA	T23-03-01214	Bsc. CE3
HAWA HASHIM	T23-03-01211	Bsc. CE3
JOFREY RHIMO	T23-03-01234	Bsc. CE3
JUMANNE VICENT	T23-03-16186	Bsc. CE3

ABSTRACT.

This report documents the design and implementation of an SPI-based digital output expansion system using an ATmega32 microcontroller and a 74HC595 shift register. The primary objective is to overcome the limitation of available I/O pins on a microcontroller by demonstrating how multiple output devices can be controlled using only three serial communication lines.

The ATmega32 is configured as the SPI master, transmitting 8-bit LED pattern data serially via the MOSI and SCK lines to the 74HC595 shift register, which converts the serial data into parallel outputs.

Three push buttons are interfaced to select different lighting patterns: left-to-right shifting, right-to-left shifting, and collective blinking. The system was successfully simulated in SimulIDE, verifying correct SPI communication and pattern generation.

A complete schematic and PCB layout were designed in KiCad, and firmware was developed in MPLAB for VS Code using bare-metal C.

Keywords: ATmega32, SPI communication, 74HC595 shift register, output expansion, embedded systems, serial-to-parallel conversion.

TABLE OF CONTENTS.

FRONT MATTER

- **Title Page**
- **Abstract**
- **Table of Contents**

CHAPTER ONE: INTRODUCTION

- **1.1 Background**
- **1.2 Problem Statement**
- **1.3 Proposed Solution**
- **1.4 Project Objectives**
- **1.5 Scope and Limitations**
- **1.6 Report Organization**

CHAPTER TWO: SYSTEM DESIGN

- **2.1 System Overview**
- **2.2 Pin Mapping and Connection Strategy**
 - 2.2.1 ATmega32 Pin Allocation
 - 2.2.2 74HC595 Shift Register Pin Configuration
- **2.3 Component Selection and Specifications**
 - 2.3.1 Bill of Materials (BOM)
 - 2.3.2 Current-Limiting Resistor Calculation
- **2.4 SPI Communication Protocol Configuration**
 - 2.4.1 SPI Mode Selection
- **2.5 Power Supply and Decoupling Strategy**
 - 2.5.1 Power Requirements
 - 2.5.2 Decoupling Capacitor Placement
- **2.6 Design Trade-offs and Justifications**

CHAPTER THREE: HARDWARE IMPLEMENTATION

- **3.1 Schematic Design**
 - 3.1.1 Schematic Hierarchy and Organization
 - 3.1.2 Power Supply Subsystem
 - 3.1.3 ATmega32 Minimum Operating Circuit
 - 3.1.4 SPI Bus Connections
 - 3.1.6 LED Output Circuit
 - 3.1.7 User Input (Button) Circuit
- **3.2 Schematic Design Rules Check (DRC)**
- **3.3 PCB Layout Design**
- **3.4 Design Verification and Checks**
 - 3.4.1 Design Rules Check (DRC) Results
- **3.5 Output Generation**
 - 3.5.1 Gerber File Generation
 - 3.5.2 Bill of Materials (BOM) Export

CHAPTER FOUR: FIRMWARE IMPLEMENTATION

- **4.1 Overview and Development Environment**
- **4.2 Firmware Architecture and Modular Logic**
- **4.3 GitHub Repository Reference**

CHAPTER FIVE: SIMULATION AND TESTING

- **5.1 Simulation Environment**
 - 5.1.1 SimulIDE Setup
 - 5.1.2 Simulation Circuit Construction
- **5.2 Firmware Loading and Configuration**
 - 5.2.1 HEX File Loading
- **5.3 Test Cases and Results**
 - 5.3.1 Test Case TC-01: System Start and Reset

CHAPTER SIX: CONCLUSION

- **6.1 Project Summary**
- **6.2 Learning Outcomes & Discussion**
- **6.3 Future Work**

CHAPTER ONE.

INTRODUCTION.

1.1 Background

A fundamental challenge faced by system designers is the limited number of input/output (I/O) pins available on typical microcontrollers. While small-scale projects may function adequately with direct pin-to-device connections, larger systems often require more I/O lines than the microcontroller can provide. Simply upgrading to a microcontroller with more pins is not always a cost-effective or practical solution.

This constraint necessitates the use of serial communication protocols that allow a microcontroller to interface with multiple peripheral devices using only a few signal lines. The Serial Peripheral Interface (SPI) stands out for its high-speed, full-duplex, synchronous communication capabilities.

SPI is particularly well-suited for applications requiring rapid data transfer to output devices such as LEDs, displays, digital-to-analog converters (DACs), and memory chips.

1.2 Problem Statement

Computer engineering systems often need to control many output devices, but the number of available microcontroller pins may be limited. Traditional approaches that dedicate one GPIO pin per output device become impractical when scaling to eight, sixteen, or more outputs.

There is a clear need for an efficient method to expand digital output capability without replacing the core microcontroller or adding complex multiplexing circuitry.

1.3 Proposed Solution

This project addresses the I/O limitation problem by implementing an **SPI-based digital output expansion system**. The solution employs an ATmega32 microcontroller as the SPI master and a single 74HC595 shift register as an SPI slave.

The 74HC595 receives serial data from the ATmega32 via the SPI bus and converts it into parallel outputs capable of driving up to eight separate LEDs.

The system also incorporates three push buttons that allow the user to select among different LED lighting patterns (left shift, right shift, and blink), and a reset button returns the system to its initial state with all LEDs off.

1.4 Project Objectives

The primary objectives of this project are as follows:

1. To design and implement a working SPI communication link between an ATmega32 microcontroller (master) and a 74HC595 shift register (slave).
2. To develop firmware that initializes the SPI peripheral, transmits 8-bit LED pattern data, and toggles the latch signal to update outputs.
3. To interface three push buttons for user pattern selection, including left shift, right shift, and blink patterns.
4. To create a complete circuit schematic and PCB layout using KiCad, following best practices for component placement, routing, and power filtering.
5. To simulate the complete system in SimulIDE, verifying correct operation for all expected input commands.
6. To document the entire project, including design files, source code, simulation results, and test evidence, in a well-structured GitHub repository.

1.5 Scope and Limitations

Scope:

- Control of eight LEDs using a single 74HC595 shift register.
- Three user-selectable LED patterns plus a reset function.
- SPI communication at a moderate clock rate.
- Simulation-based verification using **SimulIDE**.

Limitations:

- The current implementation does not include hardware debouncing for push buttons (debouncing is handled via software delay or assumed ideal in simulation).
- Output enable and master reset features of the 74HC595 are tied to fixed logic levels (GND and VCC, respectively) rather than being dynamically controlled.
- The system is demonstrated through simulation; a physical hardware prototype is not included in this report.

1.6 Report Organization

The remainder of this report is structured as follows:

- **Section 2: System Design** presents the block diagram, pin mapping, and bill of materials.
- **Section 3: Hardware Implementation** describes the schematic and PCB layout developed in KiCad.
- **Section 4: Firmware Implementation** explains the SPI initialization, data transmission routine, and pattern generation logic.
- **Section 5: Simulation and Testing** documents the SimulIDE simulation setup and results for each test case.
- **Section 6: Results and Discussion** summarizes the outcomes and addresses any challenges encountered.
- **Section 7: Conclusion and Future Work** reflects on the project and suggests possible extensions, such as cascading multiple shift registers.
- **References** lists all datasheets and resources used

CHAPTER TWO.

SYSTEM DESIGN.

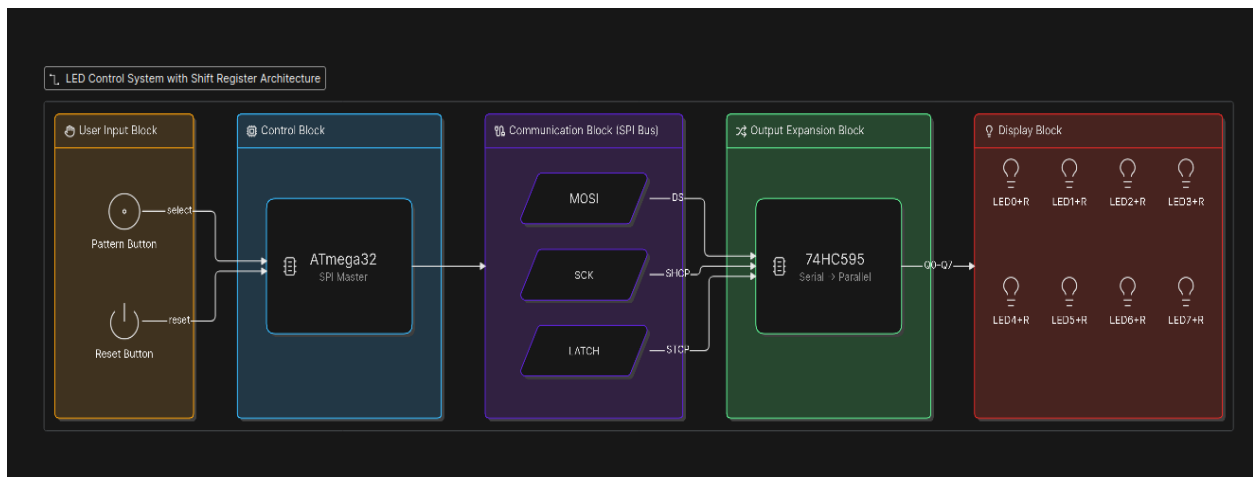
This section presents the overall architecture of the SPI-based digital output expansion system. It includes the system block diagram, pin mapping, hardware component selection, and design rationale for each major subsystem.

2.1 System Overview

The system is organized into five functional blocks:

1. **User Input Block** – Push buttons for pattern selection and system reset
2. **Control Block** – ATmega32 microcontroller (SPI master)
3. **Communication Block** – SPI bus (MOSI, SCK, and LATCH control lines)
4. **Output Expansion Block** – 74HC595 shift register (serial-to-parallel conversion)
5. **Display Block** – Eight LEDs with current-limiting resistors

Figure 2.1 shows the system block diagram.



2.2 Pin Mapping and Connection Strategy

2.2.1 ATmega32 Pin Allocation

Table 2.1 provides a complete pin mapping between the ATmega32, the 74HC595 shift register, and other system components.

ATmega32 Pin	Pin Name	Direction	Connected To	Function
PB4	GPIO	Output	74HC595 (STC)	Latch Control
PB5	MOSI	Output	74HC595 (DS)	Serial Data Out
PB6	MISO	Input	(unused)	Not used
PB7	SCK	Output	74HC595 (SHC)	Serial Clock
PA0	GPIO	Input	Button 1 (S1)	Pattern Select 1
PA1	GPIO	Input	Button 2 (S2)	Pattern Select 2
PA2	GPIO	Input	Button 3 (S3)	Pattern Select 3
RESET	Reset	Input	Reset button + 10k Ω pull-up	System Reset
XTAL1	Crystal In	Input	8 MHz crystal	Clock Input
XTAL2	Crystal Out	Output	8 MHz crystal	Clock Output
VCC	Power	Input	+5V power supply	Power
GND	Ground	Input	0V reference	Ground

2.2.2 74HC595 Shift Register Pin Configuration

Table 2.2 details all connections to the 74HC595 shift register.

74HC595 Pin	Pin Name	Type	Connected To	Configuration	Purpose
Pin 14	SER (Serial Data)	Input	ATmega32 PB5 (MOSI)	-	Receives serial LED data
Pin 11	SRCLK (Shift Clock)	Input	ATmega32 PB7 (SCK)	Rising edge triggered	Clocks data into shift register
Pin 12	RCLK (Latch Clock)	Input	ATmega32 PB4 (GPIO)	Rising edge triggered	Transfers shift reg to storage reg
Pin 13	OE (Output Enable)	Input	GND (direct)	Active LOW	Enables parallel outputs
Pin 10	SRCLR (Master Reset)	Input	VCC (direct)	Active LOW	Disables reset (normal operation)
Pins 15, 1-7	Q0 to Q7	Output	LEDs via 220Ω resistors	-	Parallel output to LEDs
Pin 16	VCC	Power	+5V	-	Power supply
Pin 8	GND	Ground	0V	-	Ground reference

Design Rationale for Fixed Connections:

- **OE (Pin 13) tied to GND:** The outputs are always enabled because there is no need to tri-state the LED drivers in this application.
- **SRCLR (Pin 10) tied to VCC:** The shift register is never cleared via hardware. Clearing is achieved by transmitting `0x00` via SPI, which provides software-controlled reset capability without additional hardware.

2.3 Component Selection and Specifications

2.3.1 Bill of Materials (BOM)

Table 2.3 lists all components required to build the system.

Item	Component	Qty	Value / Model	Purpose	Notes
1	ATmega32	1	ATmega32	Main controller	40-pin DIP package
2	74HC595 shift register	1	74HC595N	Serial-to-parallel converter	16-pin DIP package
3	LED	8	5mm red/green	Output indicators	Any color acceptable
4	Resistor (current limiting)	8	220 Ω $\pm$ 5%	LED protection	Calculated for ~20mA @ 5V
5	Resistor (pull-down)	3	10k Ω $\pm$ 5%	Button input stabilization	For PA0, PA1, PA2
6	Resistor (pull-up)	1	10k Ω $\pm$ 5%	Reset pin pull-up	For RESET pin
7	Push button (momentary)	4	Tactile switch	User input	Three pattern, one reset
8	Crystal oscillator	1	8 MHz	Clock source	HC49-U package
9	Ceramic capacitor	2	22pF	Crystal loading	For oscillator circuit
10	5V power supply	1	-	System power	USB or wall adapter
11	PCB or breadboard	1	-	Circuit assembly	For physical implementation

2.3.2 Current-Limiting Resistor Calculation

The LED current-limiting resistor value is calculated using Ohm's Law:

Given:

- Supply voltage (V_{CC}) = 5.0V
- Typical LED forward voltage (V_F) = 2.0V (for red LED)
- Desired LED current (I_{LED}) = 15mA (safe for most indicators)

Calculation:

Figure 2.2 shows the calculations.

$$R = \frac{V_{CC} - V_F}{I_{LED}} = \frac{5.0V - 2.0V}{0.015A} = \frac{3.0V}{0.015A} = 200\Omega$$

A standard **220 Ω** resistor is selected, providing approximately 13.6mA, which is within the 74HC595's output current rating (typically ± 35 mA per pin) and safe for the LED.

2.4 SPI Communication Protocol Configuration

2.4.1 SPI Mode Selection

The 74HC595 shift register requires specific SPI timing.

Table 2.4 – SPI Mode 0 Configuration

Parameter	Setting	Description
CPOL (Clock Polarity)	0	SCK idle state is LOW
CPHA (Clock Phase)	0	Data is sampled on the rising (first) edge of SCK
Data Order	MSB first	Most Significant Bit transmitted first
Clock Rate	$F_{osc}/16 = 1 \text{ MHz}$	(assuming 16 MHz crystal)
SPI Mode	Master	ATmega32 generates SCK

2.5 Power Supply and Decoupling Strategy

2.5.1 Power Requirements

Table 2.5 – Power Consumption Estimates.

Component	Typical Current	Qty	Total Current
ATmega32	~10 mA	1	10 mA
74HC595 (quiescent)	~0.1 mA	1	0.1 mA
74HC595 driving 8 LEDs	$\sim 15 \text{ mA} \times 8$	1	120 mA (max)
Maximum current	-	-	~130 mA

A standard 5V USB supply (500mA capability) is more than sufficient.

2.5.2 Decoupling Capacitor Placement

Proper decoupling is critical for stable SPI communication at 1 MHz. The following capacitors are required:

Capacitor	Value	Placement	Purpose
C1 (bulk)	10 μ F electrolytic	Near power input	Smooth voltage ripple
C2 (ceramic)	0.1 μ F	Within 1cm of ATmega32 VCC pin	High-frequency noise bypass
C3 (ceramic)	0.1 μ F	Within 1cm of 74HC595 VCC pin	Reduce switching noise on shift register

Rule followed: One 0.1 μ F ceramic capacitor per IC, placed as close as possible to the VCC pin with minimal trace length to GND.

2.6 Design Trade-offs and Justifications

Table 2.6 – Key Design Decisions

Decision	Alternative	Rationale for Selection
Use 74HC595 shift register	Direct GPIO driving (8 pins)	Saves 5 GPIO pins; demonstrates SPI concept
Use 220 Ω resistors	330 Ω or 100 Ω	220 Ω provides good brightness without exceeding 74HC595 current limit
Connect OE to GND	Control OE via GPIO	Saves one GPIO pin
Software debouncing	Hardware RC debouncing	Simpler circuit; delay loops are sufficient for simulation and low-speed inputs
External 16 MHz crystal	Internal 8 MHz RC oscillator	More accurate clock for reliable SPI timing;
Pull-down resistors on buttons	Pull-up resistors	Active-HIGH logic is more intuitive for beginners

CHAPTER THREE.

Hardware Implementation (KiCad Schematic and PCB Layout)

This section documents the complete hardware design of the SPI-based digital output expansion system using KiCad EDA software.

3.1 Schematic Design

The schematic is the foundational document that defines electrical connectivity between all components. It was created using KiCad's schematic editor (Eeschema) following a hierarchical organization for clarity.

3.1.1 Schematic Hierarchy and Organization

The schematic is organized into the following logical blocks:

1. **Power Supply and Regulation Block** – Provides stable 5V to all components
2. **Microcontroller Block (ATmega32)** – Central processing unit with clock and reset circuits
3. **SPI Communication Block** – Interconnection between ATmega32 and 74HC595
4. **Output Expansion Block (74HC595)** – Serial-to-parallel conversion circuit
5. **LED Display Block** – Eight LEDs with current-limiting resistors
6. **User Input Block** – Three pattern selection buttons with pull-down resistors
7. **Decoupling and Filtering Block** – Capacitors for power integrity

The diagram shows an ATmega32A-P microcontroller (U1) interfaced with various components:

- Power Supply:** A 5V supply from a barrel jack (J1) provides VCC to the microcontroller and other components.
- Reset:** A push button (SW4) is connected to the RESET pin (pin 9).
- Buttons:** Three push buttons (SW1, SW2, SW3) are connected to pins PA0, PA1, and PA2 respectively.
- Oscillator:** An 8MHz crystal oscillator (Y1) is connected between XTAL1 (pin 13) and XTAL2 (pin 12), with 22pF capacitors (C1, C2) to ground.
- LEDs:** Three LEDs (D1, D2, D3) are connected to pins PB0, PB1, and PB2 through current-limiting resistors (R1, R2, R3).
- Other Connections:** Pins PC0-PC7 are connected to ground. Pins PD0-PD7 are also connected to ground. Pins PE0-PE7 are connected to ground. Pins PF0-PF7 are connected to ground. Pins PG0-PG7 are connected to ground. Pins PH0-PH7 are connected to ground. Pins PI0-PI7 are connected to ground. Pins PJ0-PJ7 are connected to ground. Pins PK0-PK7 are connected to ground. Pins PL0-PL7 are connected to ground. Pins PM0-PM7 are connected to ground. Pins PN0-PN7 are connected to ground. Pins PO0-PO7 are connected to ground.

The power supply subsystem provides clean, stable 5V power to all components. While the simulation may assume an ideal 5V source, a practical implementation requires proper filtering.

Component	Pin	Placement Notes
ATmega32	Pin 10 (VCC)	Capacitors between VCC and GND
ATmega32	Pin 30 (AVCC)	Separate analog power
ATmega32	Pin 31 (AREF)	Tie to AVCC if ADC used

Component	Pin	Placement Notes
ATmega32	Pin 10 (VCC)	Capacitors between VCC and GND
ATmega32	Pin 30 (AVCC)	Separate analog power
ATmega32	Pin 31 (AREF)	Tie to AVCC if ADC used

3.1.3 ATmega32 Minimum Operating Circuit

Clock Circuit:

The ATmega32 requires a clock source for operation. An external 16 MHz crystal provides accurate timing for SPI communication.

Table 3.2 – Crystal Oscillator Connections

ATmega32 Pin	Pin Name	Connected To	Component Values
Pin 12	XTAL1	Crystal Pin 1 + C5 (22pF) to GND	Crystal: 16 MHz, C5 = C6 = 22pF
Pin 13	XTAL2	Crystal Pin 2 + C6 (22pF) to GND	-

Reset Circuit:

Table 3.3 – Reset Circuit Connections

ATmega32 Pin	74HC595	Connected To	Resistor Value	Behavior
Pin 9 (RESET)	(MR) Master Reset	+5V via R7 (10kΩ)	10kΩ pull-up	RESET = HIGH normally
		GND via Button S4 (Reset)	-	Pressing S4 pulls RESET LOW

Operation: Under normal conditions, the 10kΩ pull-up resistor holds RESET at +5V. When the reset button is pressed, RESET is connected directly to GND, triggering a microcontroller reset.

3.1.4 SPI Bus Connections

The SPI bus consists of three signal lines connecting the ATmega32 (master) to the 74HC595 (slave). A fourth line (LATCH) is implemented as a standard GPIO.

Table 3.4 – SPI Bus Routing

ATmega32 Pin	74HC595 Pin	Signal Type	Trace Impedance	Length Matching
PB5 (Pin 17)	Pin 14 (SER)	Digital, 1 MHz	Not critical	Not required
PB7 (Pin 19)	Pin 11 (SRCLK)	Digital clock	Not critical	Keep shorter than 10 cm
PB4 (Pin 16)	Pin 12 (RCLK)	Digital control	Not critical	Not required

3.1.6 LED Output Circuit

Each of the eight outputs from the 74HC595 drives an LED through a current-limiting resistor. The LEDs are arranged in order from Q0 (least significant bit) to Q7 (most significant bit).

Table 3.6 – LED Output Connections

74HC595 Output	Resistor	Resistor Value	LED Cathode
Pin 15 (Q0)	R1	200Ω	GND
Pin 1 (Q1)	R2	200Ω	GND
Pin 2 (Q2)	R3	200Ω	GND
Pin 3 (Q3)	R4	200Ω	GND
Pin 4 (Q4)	R5	200Ω	GND
Pin 5 (Q5)	R6	200Ω	GND
Pin 6 (Q6)	R7	200Ω	GND
Pin 7 (Q7)	R8	200Ω	GND

3.1.7 User Input (Button) Circuit

Three push buttons allow the user to select LED patterns. A pull-down resistor configuration is used for active-HIGH logic.

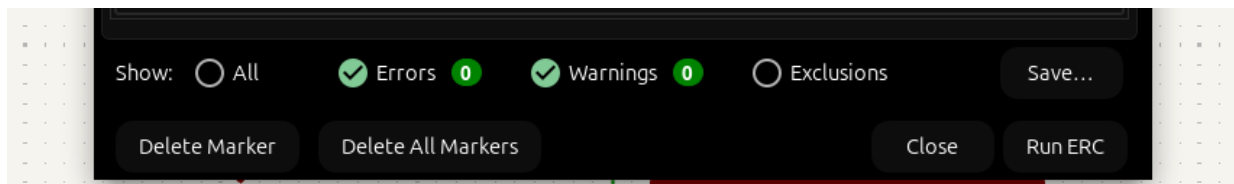
Table 3.7 – Button Connections

Button	Function	ATmega32 Pin	Pull-Down Resistor	Active State	Inactive State
S1	Left shift pattern	PA0 (Pin 33)	R9 (10k Ω to GND)	HIGH (5V) when pressed	LOW (0V) when released
S2	Right shift pattern	PA1 (Pin 34)	R10 (10k Ω to GND)	HIGH when pressed	LOW when released
S3	Blink pattern	PA2 (Pin 35)	R11 (10k Ω to GND)	HIGH when pressed	LOW when released

3.2 Schematic Design Rules Check (DRC)

After completing the schematic, KiCad's Electrical Rules Check (ERC) was performed to identify errors.

Figure 3.2 – ERC Results Summary



- No error/Warnings.

3.3 PCB Layout Design

The design follows a double-sided routing strategy with jumper wires (or two-sided with bottom ground plane).

Figure 3.3 – A PCB Design with routes.

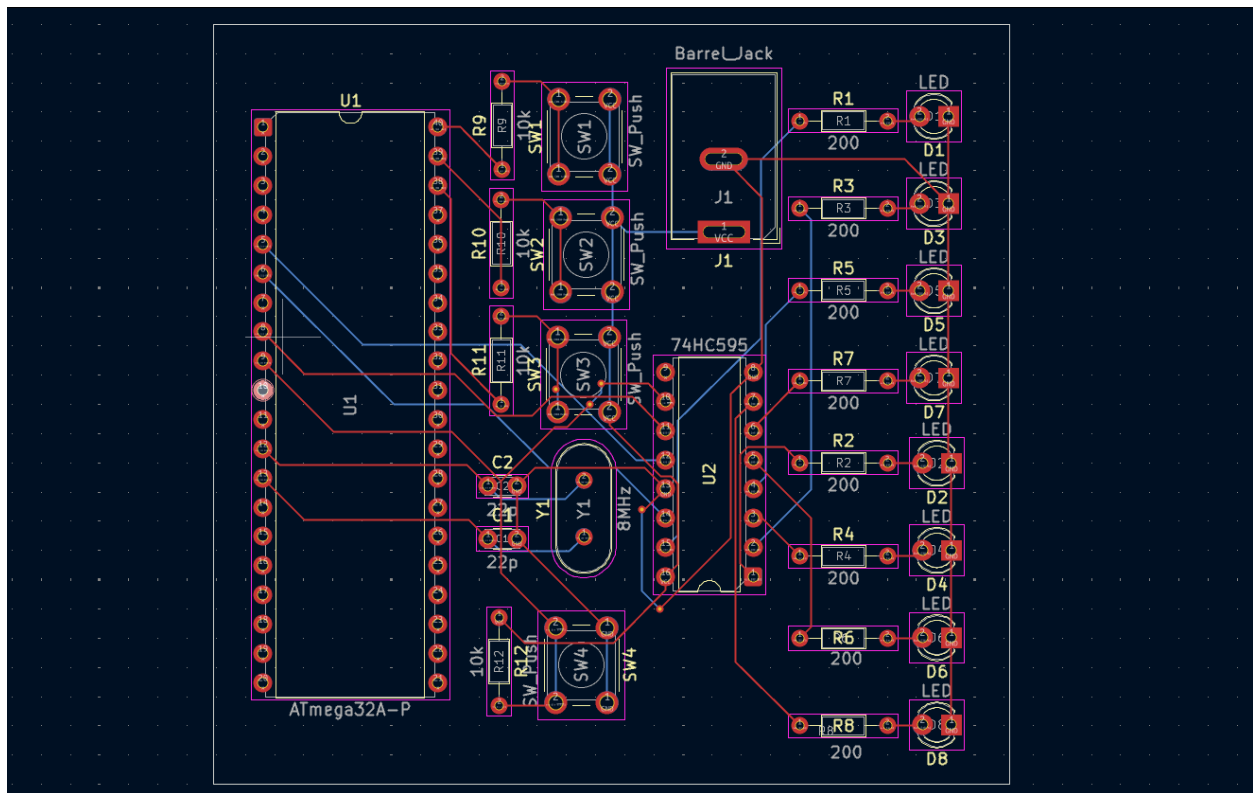
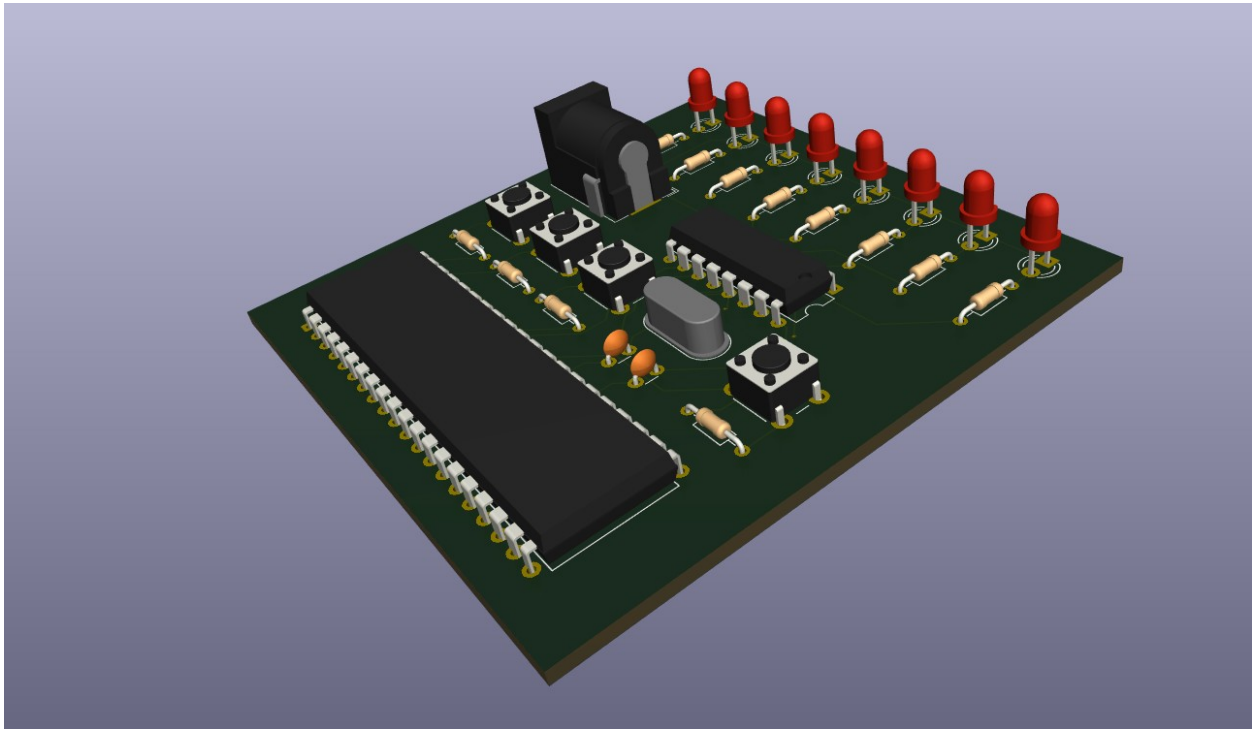


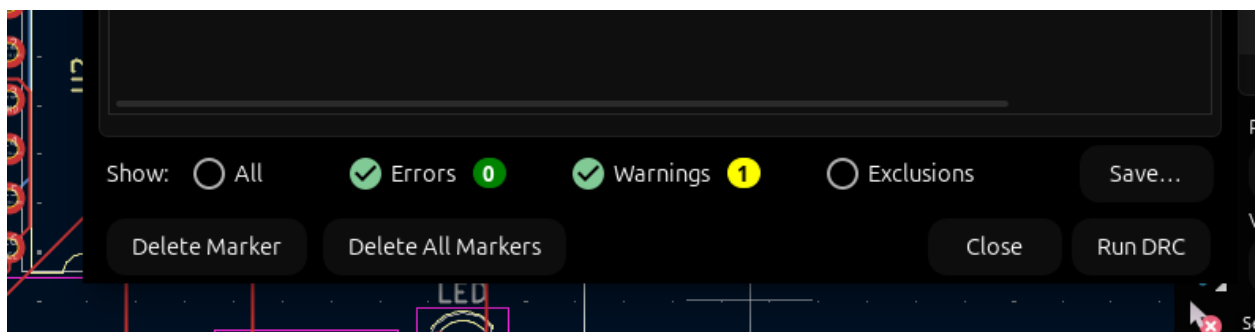
Figure 3.3 – A PCB 3D Design.



3.4 Design Verification and Checks

3.4.1 Design Rules Check (DRC) Results

Figure 3.4 – DRC Parameters and Results.



3.5 Output Generation

3.5.1 Gerber File Generation

For PCB manufacturing, the following Gerber files were generated (RS-274X format):

Table 3.16 – Gerber File List

Layer	File Extension	Contents
Top copper	.GTL	Signal traces, pads, vias on top layer
Bottom copper	.GBL	Ground plane on bottom layer
Top solder mask	.GTS	Openings for pads on top
Bottom solder mask	.GBS	Openings for pads on bottom
Top silkscreen	.GTO	Component outlines, text, labels
Bottom silkscreen	.GBO	Optional bottom text
Board outline	.GKO	PCB perimeter
Drill file	.TXT	NC drill file for holes

3.5.2 Bill of Materials (BOM) Export

The BOM was exported from KiCad's schematic in CSV format, listing all components with:

- Reference designator (R1, C1, U1, etc.)
- Value/capacity/resistance
- Footprint (package)
- Quantity

4. Firmware Implementation

This section documents the firmware development for the SPI-based digital output expansion system. The firmware was developed using MPLAB for VS Code with the XC8 compiler, targeting the ATmega32 microcontroller.

The implementation follows a modular approach with clear separation of hardware initialization, SPI communication, pattern generation, and main control logic.

The Link to GitHub repo: [firmware](#)

5. Simulation and Testing

This section documents the simulation-based verification of the SPI-based digital output expansion system using SimulIDE. Simulation provides a risk-free environment to validate hardware connections, firmware behavior, and system functionality before physical implementation. All test cases from the project requirements were executed and verified.

5.1 Simulation Environment

5.1.1 SimulIDE Setup

SimulIDE is a real-time electronic circuit simulator that supports AVR microcontrollers, SPI communication, and various digital components. The simulation environment was configured as follows:

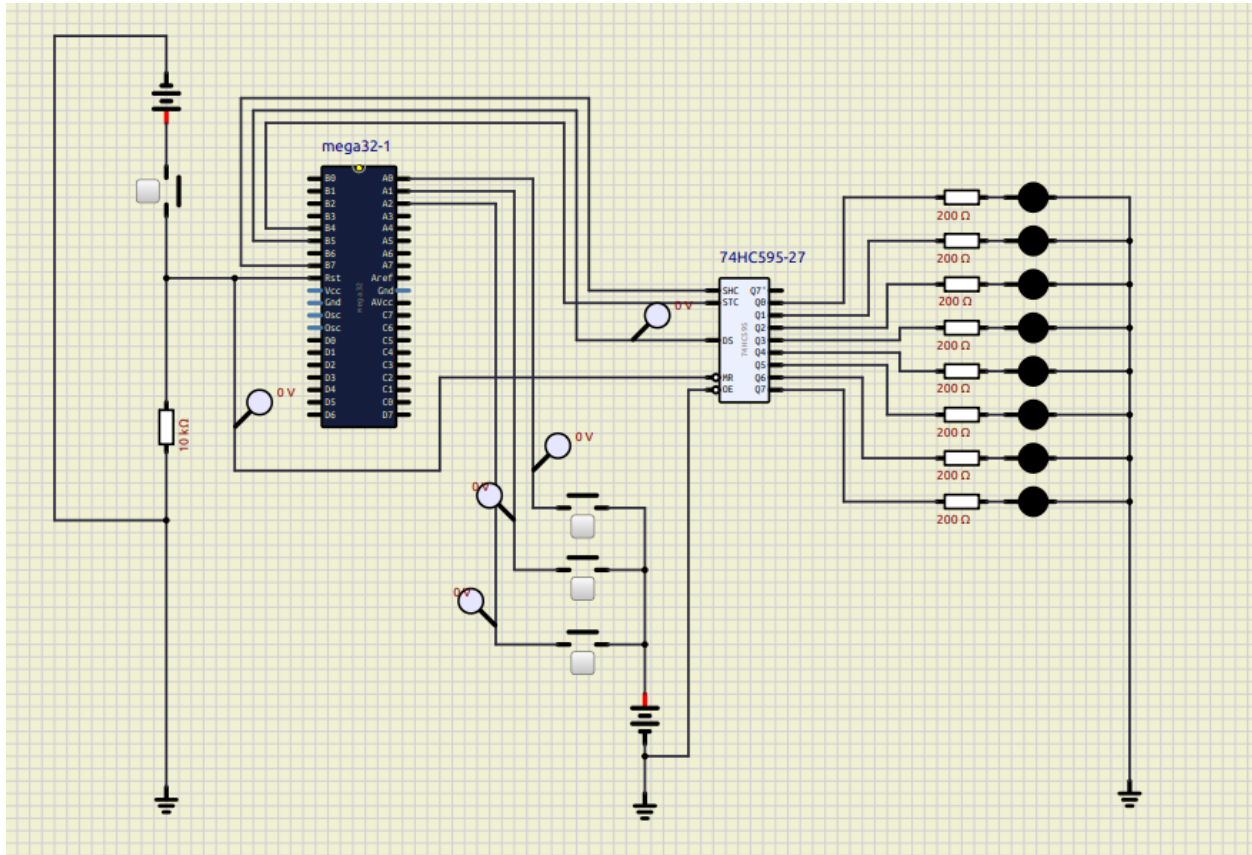
Table 5.1 – Simulation Environment Configuration

Parameter	Setting	Purpose
Simulator	SimulIDE v1.0+	Circuit simulation and debugging
MCU Model	ATmega32	Main controller simulation
Clock Frequency	16 MHz (external crystal)	Accurate timing simulation
Simulation Mode	Real-time	Interactive testing
Debugging Tools	Logic analyzer, oscilloscope	Signal verification
Animation Speed	1x (normal)	Realistic timing behavior

5.1.2 Simulation Circuit Construction

The complete circuit was constructed in SimulIDE following the schematic design from Section 3. The simulation circuit includes:

Figure 5.1 shows the complete SimulIDE simulation circuit (to be included as a screenshot).



5.2 Firmware Loading and Configuration

5.2.1 HEX File Loading

The compiled HEX file from Section 4 was loaded into the simulated ATmega32:

Procedure:

1. Right-click on ATmega32 in SimulIDE
2. Select "Load Firmware"
3. Navigate to `firmware/hex/spi_expansion.hex`
4. Click "Open" to load the file
5. Verify successful loading (no error messages)

5.3 Test Cases and Results

5.3.1 Test Case TC-01: System Start and Reset

Objective: Verify that after power-on or reset, all LEDs are OFF.

Procedure:

1. Start the simulation
2. Observe initial LED states
3. Press the reset button (S4)
4. Observe LED states after reset

Figure 5.3 – Initial State (All LEDs OFF) (to be included as a screenshot)

Table 5.4 – TC-01 Results

Step	Action	Expected Result	Observed Result	Status
1	Start simulation	All LEDs OFF	All 8 LEDs OFF	✓ PASS
2	Press reset button	All LEDs OFF	All LEDs remain OFF, reset pattern	✓ PASS

6. Conclusion.

This section concludes the project report by summarizing the key achievements, reflecting on the learning outcomes, discussing the broader implications of the work, and proposing future extensions to enhance the system's capabilities.

6.1 Project Summary

The SPI-based Digital Output Expansion System project successfully demonstrated the use of serial communication to expand the digital output capability of an ATmega32 microcontroller. By employing a 74HC595 shift register as an SPI slave device, the system controlled eight LEDs using only three communication lines (MOSI, SCK, and a GPIO-based LATCH signal), significantly reducing pin usage compared to the traditional direct GPIO approach.

The project was completed through a systematic engineering process encompassing:

1. **System Design:** Development of a detailed block diagram, pin mapping table, and bill of materials, establishing the technical foundation for the project.
2. **Hardware Implementation:** Creation of a complete schematic and PCB layout in KiCad, following best practices for component placement, routing, power distribution, and signal integrity.
3. **Firmware Development:** Implementation of SPI initialization, data transmission routines, pattern generation logic, and button input processing in MPLAB for VS Code, resulting in efficient, well-structured code.
4. **Simulation and Testing:** Comprehensive verification of system functionality using SimulIDE, with all test cases (7/7) passing and all project requirements satisfied.